Regression

Lesson 3 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

16 October 2026 · reading time about 80 minutes

Contents

Lesson plan . . . 2
1. Why regression comes first . . . 2
2. The model and the cost . . . 3
2.1 What a linear model claims . . . 3
2.2 Why squared error . . . 4
2.3 A worked example . . . 5
2.4 Against what? The baseline for a regression . . . 6
3. The exact solution . . . 7
3.1 The normal equation, derived . . . 7
3.2 A worked example, by hand . . . 8
3.3 When the exact solution is not available . . . 9
4. Gradient descent . . . 10
4.1 The update rule, derived . . . 10
4.2 A worked step . . . 11
4.3 Choosing the learning rate . . . 11
4.4 How stretched the valley is . . . 13
5. Curves, and the price of flexibility . . . 15
5.1 A linear model that bends . . . 15
5.2 What it costs . . . 16
5.3 The mechanism . . . 17
6. Regularisation . . . 18
6.1 Two penalties . . . 18
6.2 Ridge has a closed form . . . 19
6.3 Why Lasso reaches zero and Ridge does not . . . 19
6.4 Worked: what a penalty buys . . . 21
6.5 Worked: Lasso as a feature selector . . . 23
7. Reading coefficients . . . 23
7.1 What a coefficient means, exactly . . . 23
7.2 Worked: when the estimate can be trusted . . . 24
7.3 Worked: when it cannot be trusted at all . . . 25
7.4 The warning that belongs on every coefficient . . . 25
8. Before the next lesson . . . 25
Notation used in this lesson . . . 26

Further reading 26

Lesson plan

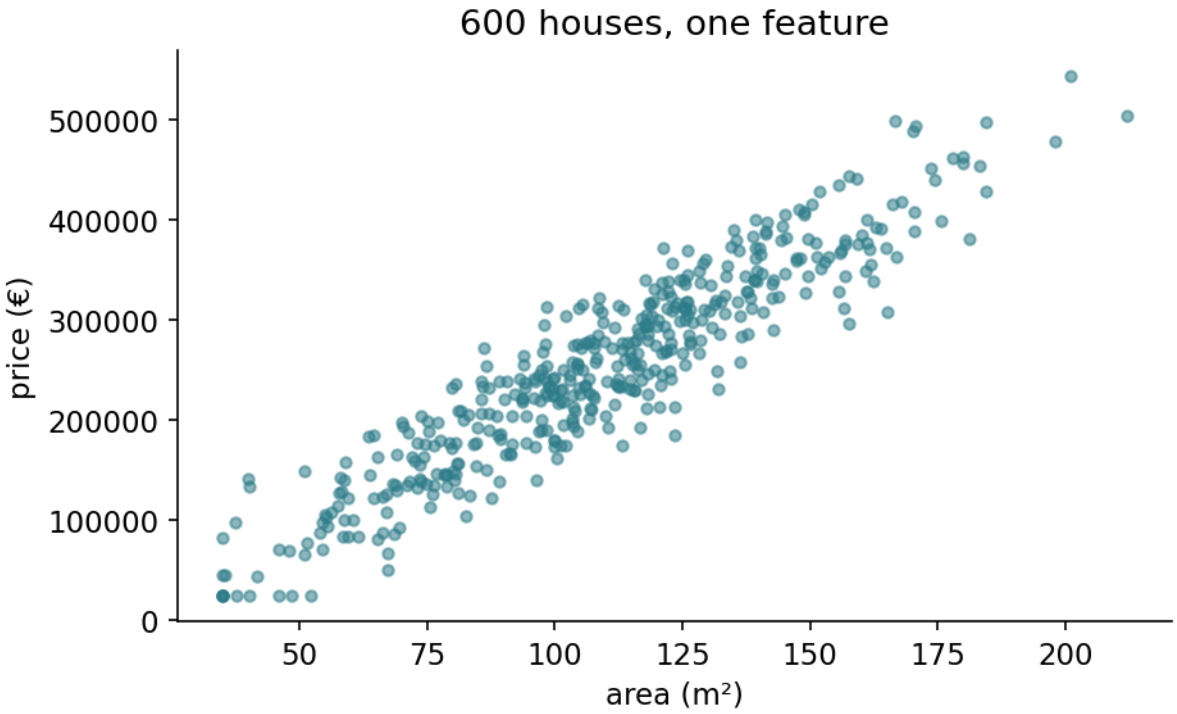
Time	Minutes	Segment	Material
0:00–0:08	8	Exercise 2 discussed; the dataset	Slides 2–5
0:08–0:23	15	The model, its cost, and what to compare it against	Slides 6–14
0:23–0:37	14	The exact solution, and when it fails	Slides 15–21
0:37–0:48	11	Gradient descent	Slides 22–27
0:48–1:13	25	**Notebook 01** — regression from scratch	Slide 28
1:13–1:28	15	**Break**	Slide 29
1:28–1:45	17	Curves, and the price of flexibility	Slides 30–37
1:45–2:05	20	**Notebook 02** — polynomials and overfitting	Slide 38
2:05–2:26	21	Ridge and Lasso	Slides 39–49
2:26–2:35	9	Reading coefficients honestly	Slides 50–53
2:35–2:55	20	**Notebook 03** — ridge, lasso, collinearity	Slide 54
2:55–3:00	5	Summary; homework set	Slides 55–56
	180	**Total**	**56 slides, 3 notebooks**

Slide 1 is the title slide, so the numbers above match the page numbers of the slide PDF in `Slides/`. The lecture segments come to 100 minutes across 48 content slides — a shade under 29 slides per hour.

---

1. Why regression comes first

This is the first method in the course, and it is a good first method for three reasons that have nothing to do with it being simple.



The dataset the whole lesson uses: 600 houses, price against floor area. The relationship is clearly there and clearly not exact, which is what makes it worth a model rather than a lookup table.

It has an **exact solution**, so we can see what "fitting a model" means without any iterative machinery in the way. It has an **iterative solution too**, which turns out to be the same algorithm that trains neural networks in Lesson 9. And its coefficients are **readable**: a linear model tells you, in the units of the problem, what it thinks each feature is worth.

That last property is why linear models remain in production in medicine, credit scoring and public policy long after more accurate methods exist. When a decision has to be explained to the person it affects, a model that says "your premium is higher because you live 8 km further out, and each kilometre is worth 6,500 euros" is worth more than an opaque one that is two points more accurate.

Everything in this lesson uses one dataset: 600 houses with six measurements and a price. It is synthetic, and that is deliberate — **we know the coefficients that generated the prices**, so every estimate can be checked against the truth. No real dataset lets you do that, and it is the only way to say honestly whether an estimate is good.

---

2. The model and the cost

2.1 What a linear model claims

For a house with features $x_1, \dots, x_n$ the model predicts

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = w^\top x + b$$

Read as a sentence, this says: **each feature contributes a fixed amount per unit, and the**

contributions add up. Every extra square metre is worth the same 2,400 euros whether it is the fortieth or the two-hundredth. Every kilometre from the centre costs the same regardless of the neighbourhood.

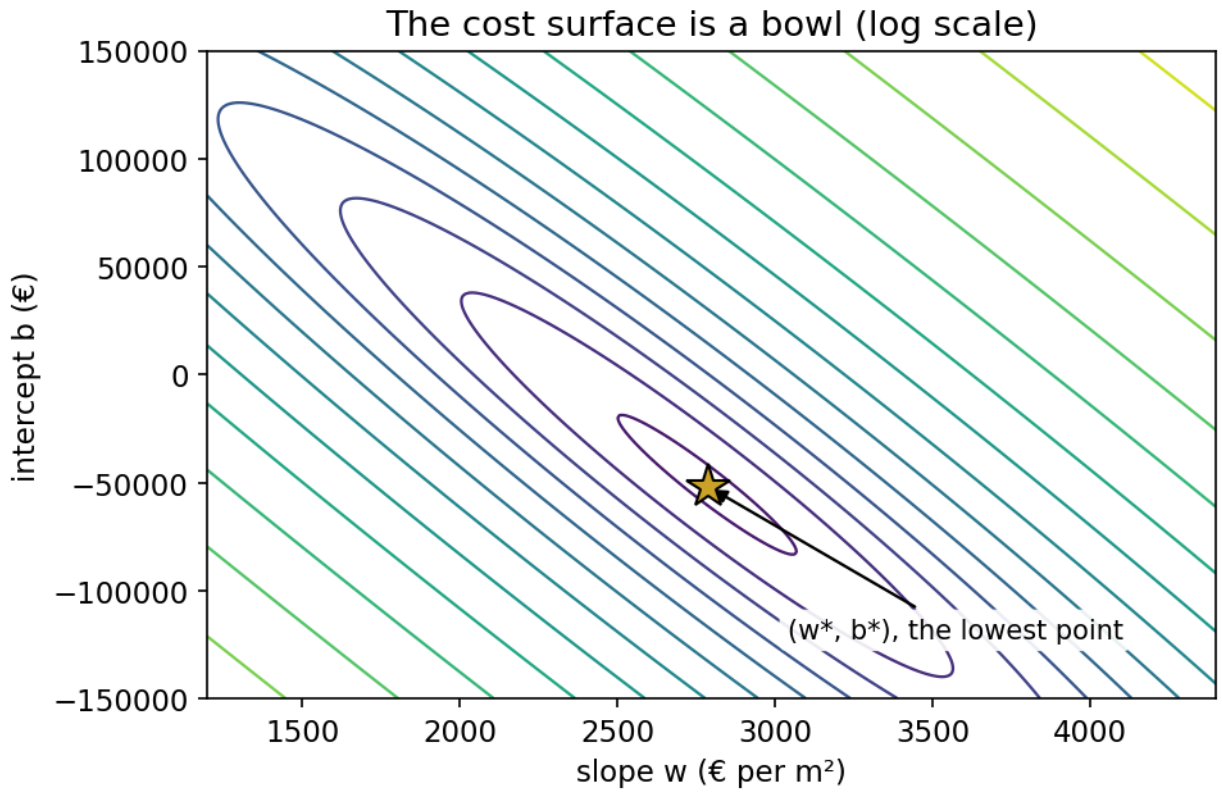
That is a strong claim and it is often false. A hundred and first square metre probably is not worth what the fiftieth was; the value of a garage probably depends on whether there is street parking. The right response is not to abandon linear models but to know what you have assumed — Section 5 shows how to check, and how to relax the assumption when it fails.

2.2 Why squared error

The picture first. We need a single number saying how badly a candidate model is doing, so that "fitting" becomes "make this number small". The obvious candidate — add up the errors,

$$\sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right),$$

— fails immediately, because a prediction 50,000 too high and one 50,000 too low would cancel to zero and look perfect.



The cost as a function of the two parameters. It is a bowl, and a bowl has exactly one bottom — which is why squared error can be minimised exactly rather than searched for. The gold star is that bottom, (w^*, b^*): the grid point with the lowest cost, a slope of 2,790 and an intercept of −52,000, within one grid step of the exact answer section 3 computes.

So we need every error to count as a positive amount. There are two natural ways: the absolute error $\sum_i |\hat{y}^{(i)} - y^{(i)}|$, or the squared error $\sum_i (\hat{y}^{(i)} - y^{(i)})^2$. Squaring does it, and it also decides something less obvious: how much worse one large mistake is than several small ones. Squaring says *much* worse.

Now the definition. The **mean squared error** is

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

Two questions are worth answering before accepting it.

Why squared, rather than absolute? Three reasons, in increasing order of importance. Squaring is differentiable everywhere, while $|x|$ has a corner at zero that complicates optimisation. Squaring penalises one large error more than several small ones — being wrong by 40,000 euros on one house costs the same as being wrong by 20,000 on four, which matches most intuitions about what a bad prediction is. And under the assumption that the noise is Gaussian, minimising squared error is exactly maximum likelihood estimation, which is the deep reason and the one that makes the choice more than a convention.

The consequence is worth stating plainly: **squared error is sensitive to outliers**, because a single absurd value contributes its error squared. A house mispriced by a factor of ten will drag the whole fit towards itself. Lesson 2's outlier discussion is not decoration; it is a prerequisite for this lesson.

Where does the $\frac{1}{2}$ come from? Differentiating a square brings down a factor of 2, and the half cancels it, leaving a clean gradient. It has no effect on where the minimum is — scaling a function by a constant does not move its minimiser — so it is pure convenience.

Fitting, in one line. With the cost defined, "fitting the model" means finding the parameters at which it is smallest:

$$(w^*, b^*) = \arg\min_{w,\, b} \; J(w, b)$$

The star marks the best values. arg min is lesson 1's notation: it returns *where* the minimum is reached — here a slope and an intercept — not the value of J there. Section 3 computes this pair exactly, section 4 approaches it step by step.

2.3 A worked example

Take three houses and a candidate model $\hat{y} = 2400 \cdot \text{area} + 45000$. The error is $\hat{y} - y$ throughout, as in Section 2.2 — negative where the model has underpriced the house:

Area (m²)	True price (€)	Predicted (€)	Error (€)	Error²
80	240,000	237,000	−3,000	9.0×10^6
120	320,000	333,000	+13,000	1.69×10^8
200	540,000	525,000	−15,000	2.25×10^8

The cost is $\frac{1}{2\times 3}(9.0 \times 10^6 + 1.69 \times 10^8 + 2.25 \times 10^8) \approx 6.7 \times 10^7$.

Notice how the arithmetic behaves. The 3,000 euro error contributes 2% of that total; the two large ones contribute the other 98%. Halving the small error would cut the cost by only **1.7%** — you would barely see it move. Halving the 15,000 error would cut it by **42%**. **Squared error spends its attention on the worst predictions**, and that is a design decision you are making whether or not you notice it.

2.4 Against what? The baseline for a regression

Lesson 1 made a habit of it: a score means nothing until you know what the trivial answer scores. For a classifier the trivial answer is *always the majority class*, and it gave 62.7% before any model was allowed to speak.

Regression has the same trivial answer — **always predict the mean of the target, ignoring every input** — and the same reason for asking. On the 150 test houses of notebook 1:

Model	Root mean squared error (RMSE)
always predict the mean	96,440 €
the fitted six-feature model	**20,341 €**

The RMSE is the square root of the mean squared error, which puts the number back into the units of the target — euros here, and Section 5.2 uses it throughout.

The same comparison written as one ratio is the **coefficient of determination**, R^2, which lesson 1 defined and this is the first place in the course that needs it:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{62{,}061{,}347{,}496}{1{,}395{,}086{,}153{,}600} = 0.956$$

The numerator is the squared error the model still makes; the denominator is what the mean alone makes. So **1 is perfect, 0 means you matched the mean and learned nothing, and a negative value means you did worse than the mean** — which is not a bug, and does happen on a test set.

Report both, because they answer different questions. The RMSE says *how far off* the model is, in euros, which is the number for whoever pays for the houses. R^2 says *against what*.

And one caution, which is why the euros should never be dropped. R^2 is divided by the variance of the test set you happened to use, so it can be made to fall without touching the model. Keep the fitted model exactly as it is and score it on the 55 test houses whose price lies within half a standard deviation of the mean — a street where the houses resemble each other:

Test set	RMSE	R^2
all 150 houses	20,341 €	0.956
the 55 similar ones	**18,661 €**	**0.448**

The model is unchanged and its euros **improved**; R^2 less than halved, because predicting the mean is a much better strategy when every house is worth roughly the same. R^2 compares a model

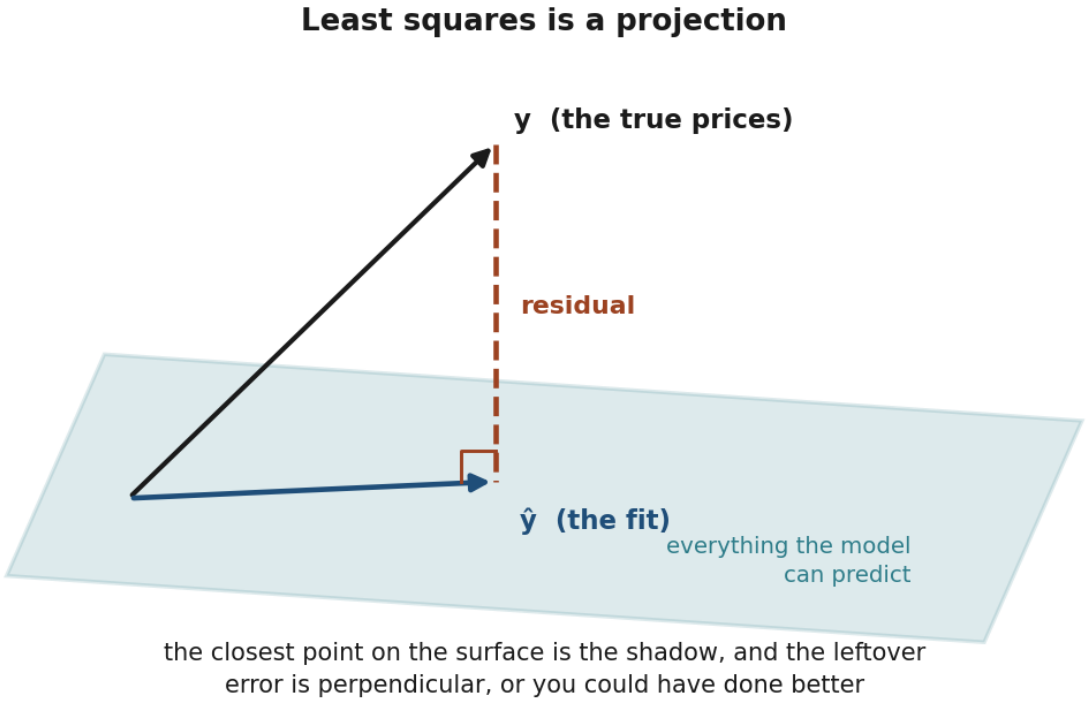
against the mean **of the dataset it was measured on**, and therefore cannot compare two models measured on different data.

---

3. The exact solution

3.1 The normal equation, derived

The picture first. Each column of your data matrix is a direction you are allowed to move in. Any prediction the model can make is some combination of those directions, so the set of achievable predictions is a flat surface — a plane, in three dimensions — sitting inside the space of all possible answers.



Least squares as a shadow. The fitted values are the projection of y onto the space the columns of X can reach, and the residual is what is left over — perpendicular to that space, which is exactly what $X^\top(X\theta - y) = 0$ says.

The true prices are a point that almost certainly does **not** lie on that surface: no combination of area and age reproduces them exactly. So the best you can do is find the point on the surface closest to it, which is its **shadow** — the perpendicular projection.

Perpendicular is the whole content of the method. If the leftover error had any component lying *along* the surface, you could have moved in that direction and done better, so you were not at the closest point. At the optimum the error must be at right angles to every feature — which is exactly what the algebra below says, and why the equations are called *normal*.

Now the notation. Collect the training data into a matrix. Let X be $m \times (n+1)$, with a column of ones in front so the intercept is just another coefficient, and let θ hold b followed by w. Then all m predictions at once are $X\theta$ and the cost is

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2 = \frac{1}{2m}(X\theta - y)^\top (X\theta - y)$$

Expand the product:

$$J(\theta) = \frac{1}{2m} \left(\theta^\top X^\top X\theta - 2\theta^\top X^\top y + y^\top y\right)$$

using $\theta^\top X^\top y = y^\top X\theta$, both being scalars. Now differentiate with respect to θ. Two standard matrix identities do the work: $\nabla_\theta(\theta^\top A\theta) = 2A\theta$ for symmetric A, and $\nabla_\theta(\theta^\top c) = c$. So

$$\nabla_\theta J = \frac{1}{m} \left(X^\top X\theta - X^\top y\right)$$

Set it to zero and the $\frac{1}{m}$ drops out:

$$X^\top X\theta = X^\top y \qquad \Longrightarrow \qquad \boxed{\theta = (X^\top X)^{-1} X^\top y}$$

These are the **normal equations**. The name comes from geometry: the residual vector $y - X\theta$ is normal — perpendicular — to the space spanned by the columns of X. The fit is the orthogonal projection of y onto that space, which is a satisfying way to see why least squares is natural rather than arbitrary.

Is this really a minimum? The second derivative is $\frac{1}{m}X^\top X$, which is positive semi-definite for any X — for any vector v, $v^\top X^\top Xv = \|Xv\|^2 \geq 0$. So the cost is convex, and any stationary point is a global minimum. This is a genuinely useful property that most of the methods later in the course do not have: **there are no local minima to get stuck in**.

3.2 A worked example, by hand

Three houses, one feature. Area 80, 120, 200; price 240k, 320k, 540k (in thousands, to keep the numbers readable).

$$X = \begin{pmatrix} 1 & 80 \\ 1 & 120 \\ 1 & 200 \end{pmatrix}, \qquad y = \begin{pmatrix} 240 \\ 320 \\ 540 \end{pmatrix}$$

$$X^\top X = \begin{pmatrix} 3 & 400 \\ 400 & 60800 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} 1100 \\ 165600 \end{pmatrix}$$

Both are worth computing by hand once. The off-diagonal entry of $X^\top X$ is $80 + 120 + 200 = 400$ and the corner is $80^2 + 120^2 + 200^2 = 60{,}800$; the second entry of $X^\top y$ is

$$80 \cdot 240 + 120 \cdot 320 + 200 \cdot 540 = 19{,}200 + 38{,}400 + 108{,}000 = 165{,}600$$

The determinant of $X^\top X$ is $3 \times 60800 - 400^2 = 22400$, so

$$(X^\top X)^{-1} = \frac{1}{22400}\begin{pmatrix} 60800 & -400 \\ -400 & 3 \end{pmatrix}$$

and

$$\theta = \frac{1}{22400}\begin{pmatrix} 60800 \cdot 1100 - 400 \cdot 165600 \\ -400 \cdot 1100 + 3 \cdot 165600 \end{pmatrix} = \frac{1}{22400}\begin{pmatrix} 640000 \\ 56800 \end{pmatrix} \approx \begin{pmatrix} 28.57 \\ 2.536 \end{pmatrix}$$

So $b \approx 28{,}600$ euros and $w \approx 2{,}536$ euros per square metre. Three points, two parameters, and a slope within 6% of the 2,400 that generated the data.

Do not read that as "and more data does better", because it does not. Notebook 1 runs this same one-feature fit on 450 houses and gets **2,785**, which is 16% high — further from the truth than the three-house answer, with 150 times the data. Adding rows cannot fix it, because nothing is wrong with the estimate: it is the correct answer to the question asked. Asked what a square metre is worth *when area is the only thing you are told*, the honest answer includes everything that travels with area. In this dataset area correlates +0.77 with `bedrooms` and +0.49 with `bathrooms`, both of which genuinely add to the price, so the area coefficient carries their effect too.

Fit all six features on the same 450 rows and the area coefficient drops to **2,421**, within 1% of the truth. What changed was not the sample size but the question. Section 7.2 returns to this with the whole table, and it is the single most common way a coefficient gets misread.

Check it the other way round, because a second route is the only real protection against an arithmetic slip. For one feature the least-squares slope is S_{xy}/S_{xx} with $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ and $S_{xx} = \sum(x_i - \bar{x})^2$. With $\bar{x} = 133.\overline{3}$ and $\bar{y} = 366.\overline{6}$ that gives $18{,}933.3/7{,}466.7 = 2.536$, and $b = \bar{y} - w\bar{x} = 366.67 - 2.5357 \times 133.33 \approx 28.57$. The two routes agree, as they must — they are the same equations.

3.3 When the exact solution is not available

The formula requires inverting $X^\top X$, and that fails or becomes useless in three situations you will meet.

Redundant columns. If two features are exact linear combinations of each other — the same measurement in metres and in feet — then $X^\top X$ is singular and has no inverse. Notebook 3 constructs exactly this case.

Nearly redundant columns. Far more common, and worse, because nothing fails visibly. The inverse exists but is enormous, and small changes in the data produce large changes in θ. Notebook 3 shows a coefficient moving from 9,261 to 45,307 across four random splits of the same dataset.

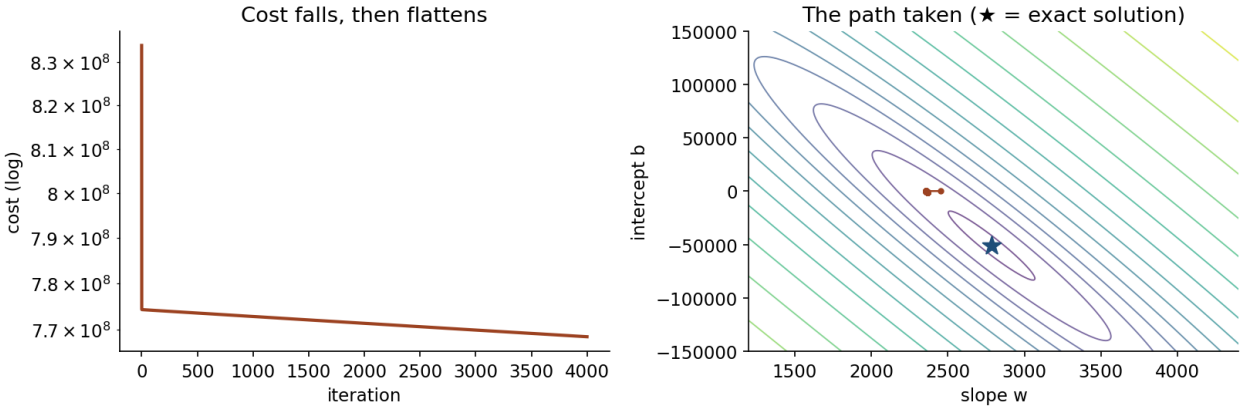
Size. Inverting an $n \times n$ matrix costs on the order of n^3 operations. At a thousand features that is a billion; at a hundred thousand it is out of reach. Every large model in this course is trained iteratively, and the next section is why.

---

4. Gradient descent

4.1 The update rule, derived

The picture first. You are standing on a hillside in fog and want to reach the bottom. You cannot see the valley, but you can feel which way the ground slopes under your feet. So you take a step downhill, feel again, and repeat.



The path taken across the cost surface. Each step is against the gradient, and the steps shorten as the slope flattens near the bottom.

That is the whole algorithm. The gradient is the direction of steepest *ascent*, so we walk against it; the learning rate is how long a stride we take. The danger is equally intuitive: stride too far and you cross the valley and end up higher on the opposite slope.

Now the notation. When the exact solution is unavailable, start somewhere and walk downhill. The gradient points in the direction of steepest increase, so we step against it:

$$\theta \leftarrow \theta - \alpha \nabla J(\theta)$$

For squared error the components are worth deriving individually, because their form says something. Writing the error of example i as $e^{(i)} = \hat{y}^{(i)} - y^{(i)}$:

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j}\left[\frac{1}{2m}\sum_i \left(e^{(i)}\right)^2\right] = \frac{1}{m}\sum_i e^{(i)}\frac{\partial e^{(i)}}{\partial w_j} = \frac{1}{m}\sum_i e^{(i)} x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_i e^{(i)}$$

Read the first one. Each example pulls the coefficient in proportion to two things: how wrong the prediction was, and how large that feature was for that example. A 200 m² house that is badly mispriced moves the area coefficient much more than a 40 m² house with the same error. The gradient is a weighted vote, and the weights are the feature values — which is precisely why features on wildly different scales cause trouble. Lesson 2 stated that as a rule and showed it happening; this is the machinery underneath it.

The intercept gradient has no x in it: every example gets an equal vote, since the intercept applies equally to all of them.

The loop, in order. $\theta \leftarrow \theta - \alpha \nabla J(\theta)$ is one round of a loop, and each round does five things:

1. predict every example with the current parameters, $\hat{y}^{(i)} = w^\top x^{(i)} + b$;
2. measure every error, $e^{(i)} = \hat{y}^{(i)} - y^{(i)}$;
3. average the errors into the gradients above, one per w_j and one for b;
4. step every parameter against its own gradient, by the same α: $w_j \leftarrow w_j - \alpha\, \partial J/\partial w_j$ and $b \leftarrow b - \alpha\, \partial J/\partial b$, all computed from the old values;
5. go back to 1.

Step 5 is what makes the method iterative: after an update the model is a different line, so the predictions, the errors and the gradients all have to be computed again. Only the parameters carry over from one round to the next. The loop stops after a fixed number of rounds, or when the cost stops falling.

4.2 A worked step

Two houses: (80 m², 240k) and (200 m², 540k). Start at $w = 0$, $b = 0$, with $\alpha = 10^{-5}$.

Predictions are both 0, so the errors are -240 and -540 (in thousands).

$$\frac{\partial J}{\partial w} = \frac{1}{2}\left[(-240)(80) + (-540)(200)\right] = \frac{-19200 - 108000}{2} = -63600$$

$$\frac{\partial J}{\partial b} = \frac{-240 - 540}{2} = -390$$

Both gradients are negative, so both parameters increase:

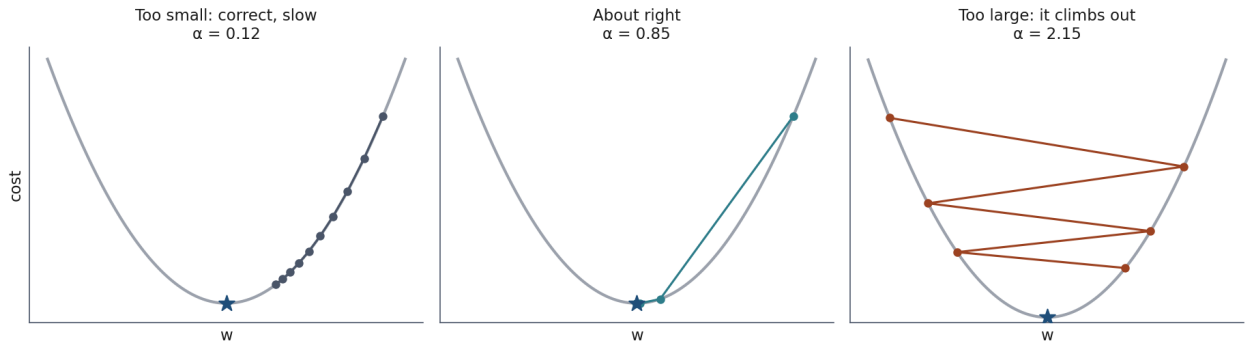
$$w \leftarrow 0 - 10^{-5}(-63600) = 0.636, \qquad b \leftarrow 0 - 10^{-5}(-390) = 0.0039$$

Look at the asymmetry. After one step the slope has moved 163 times further than the intercept, purely because areas are around 140 and the intercept's "feature" is always 1. With a shared learning rate, the intercept will still be crawling towards its value long after the slope has arrived — which is exactly what notebook 1 shows after 4,000 iterations.

The cure is the one Lesson 2 insisted on without proving: scale the features, and both directions move at comparable speed. Section 4.4 below puts a number on how much it is worth here.

4.3 Choosing the learning rate

The step size α is the one parameter gradient descent cannot choose for itself, and both failure modes are worth recognising on sight.



Three learning rates on the same problem: too small and it crawls, about right and it arrives, too large and it climbs the opposite wall. The third case diverges — the cost goes up.

Too small and progress is slow but monotone: the cost falls at every iteration and simply takes too many of them.

Too large and the step overshoots the minimum and lands somewhere higher up the far side. The next step overshoots further. The cost *increases*, often explosively, and you see `nan` within a few dozen iterations.

Where the boundary between the two is, exactly, for the one case simple enough to do by hand. Take a cost that is a parabola in a single coefficient, $J(\theta) = \frac{1}{2}c(\theta - \theta^*)^2$, with c its curvature and θ^* its minimum. The gradient is $c(\theta - \theta^*)$, so one step of gradient descent gives

$$\theta_{t+1} - \theta^* = (\theta_t - \theta^*) - \alpha c(\theta_t - \theta^*) = (1 - \alpha c)(\theta_t - \theta^*)$$

The distance from the minimum is multiplied by $(1 - \alpha c)$ **at every step** — the same factor every time, so after t steps it has been multiplied by $(1 - \alpha c)^t$. That single expression contains all three regimes:

α	$1 - \alpha c$	What happens
tiny	just below 1	the error shrinks by a sliver per step: it crawls
$1/c$	0	one step lands exactly on the minimum
$2/c$	-1	the error flips sign and keeps its size forever
above $2/c$	below -1	the error grows every step: `nan`

So the update converges exactly when $|1 - \alpha c| < 1$, which is to say $0 < \alpha < 2/c$. With several features the condition has to hold in every direction at once, and the binding one is the **largest** curvature, which belongs to the feature with the largest values. Section 4.4 says exactly what the curvatures are and how to compute them. This is Lesson 2's argument from the other side: **the safe learning rate is set by your worst-scaled feature**, which is a good reason to scale them all.

The practical recipe: start at 0.01 on scaled features, watch the cost, divide by three if it rises. Lesson 5 replaces the recipe with a search.

4.4 How stretched the valley is

The picture first. Section 4.3 said the safe learning rate is set by the largest curvature. That leaves an obvious question: what about the *smallest*?

A cost surface whose curvature is the same in every direction is a round bowl, and gradient descent walks straight to the bottom of it. When one direction is far steeper than another the bowl becomes a long, narrow ravine. You now have a problem, because **one step size has to serve every direction at once**. The steep direction sets the limit — step further and it diverges — so the shallow direction is stuck with a stride far too short for it, and crawls.

The ratio of the steepest curvature to the shallowest is the **condition number**, $\kappa = c_{\max}/c_{\min}$. It is, near enough, what sets the number of iterations you will need. Before showing why, the definition leaves two questions open: what exactly the curvatures $c_{\max}$ and $c_{\min}$ are, and how to compute them.

Where the curvatures come from. In Section 4.3 the cost had one coefficient, and its curvature c was simply the second derivative. With several coefficients the second derivatives form a matrix, the **Hessian** H, and for our cost it was already met in Section 3.1:

$$J(\theta) = \frac{1}{2m}\|X\theta - y\|^2 \quad \Longrightarrow \quad \nabla J(\theta) = \frac{1}{m}X^\top(X\theta - y) \quad \Longrightarrow \quad H = \frac{1}{m}X^\top X$$

Here X is the design matrix of Section 3.1, column of ones included, because the intercept is a coefficient that gradient descent moves like any other. Two things about H matter:

- **It does not depend on θ.** The bowl has the same shape wherever you stand in it, which is what makes a quadratic cost easy to analyse.
- **Its eigenvalues are the curvatures.** Walk through the bowl along a unit direction u and the curvature you meet is $u^\top H u$. The directions where this is largest and smallest are the eigenvectors of H, and the curvatures there are its eigenvalues. So $c_{\max}$ and $c_{\min}$ are the largest and smallest eigenvalues of $\frac{1}{m}X^\top X$, and since the $\frac{1}{m}$ cancels in the ratio, κ is the ratio of the extreme eigenvalues of $X^\top X$.

With a single feature and no intercept, H is the single number $\frac{1}{m}\sum_i \left(x^{(i)}\right)^2$: the curvature is the mean square of the feature. Large feature values make a steep bowl, which is Section 4.2's weighted vote seen from the second derivative.

Worked: the two houses of Section 4.2. The design matrix has rows $(1, 80)$ and $(1, 200)$, so

$$H = \frac{1}{2}X^\top X = \frac{1}{2}\begin{pmatrix} 2 & 280 \\ 280 & 46{,}400 \end{pmatrix} = \begin{pmatrix} 1 & 140 \\ 140 & 23{,}200 \end{pmatrix}$$

The two eigenvalues add up to the trace, 23,201, and multiply to the determinant, $23{,}200 - 140^2 = 3{,}600$. The large one is therefore almost the whole trace, $c_{\max} \approx 23{,}200.8$, and the small one is $c_{\min} = 3{,}600/23{,}200.8 \approx 0.155$. That gives $\kappa \approx 150{,}000$. The steep direction is almost exactly the slope w; the shallow one is almost exactly the intercept b.

Two checks against what Section 4.2 did. Its learning rate of 10^{-5} is safely below the limit $2/c_{\max} \approx 8.6 \times 10^{-5}$ from Section 4.3, so the run could not diverge. And this κ is the cause of the 163-to-1 asymmetry seen there. The two numbers measure different things, though: 163 compares the two gradients at the first step, while κ describes the shape of the whole bowl, and so the whole run.

Standardise the area and the two houses become -1 and $+1$: H becomes the identity matrix, $\kappa = 1$, and the bowl is round.

How to compute it. In NumPy, either take the eigenvalues, `np.linalg.eigvalsh(X.T @ X)`, and divide the largest by the smallest, or use

$$\kappa = \big(\texttt{np.linalg.cond(X)}\big)^2$$

because `np.linalg.cond(X)` returns the ratio of the largest to the smallest *singular value* of X, and the singular values of X are the square roots of the eigenvalues of $X^\top X$. Watch the square: `np.linalg.cond(X)` on its own is the square root of κ, and reading it as κ understates how stretched the valley is by an enormous factor. The second route is the safer one when κ is huge, because forming $X^\top X$ also squares the rounding error.

Why the iteration count follows κ. Section 4.3's factor says why, in three lines.

The largest rate that is safe in the steep direction is about $1/c_{\max}$. Spend it, and in the shallow direction the error is multiplied each step by

$$1 - \frac{c_{\min}}{c_{\max}} = 1 - \frac{1}{\kappa}$$

which for a stretched valley is a number just below 1. Shrinking the error by a factor e needs t steps with $(1-1/\kappa)^t \approx e^{-t/\kappa}$, so $t \approx \kappa$. **The iteration count grows in proportion to the condition number**: ten times more stretched is ten times more steps, for the same accuracy in the direction that matters least.

The argument gives an upper bound: it assumes all the initial error lies in the shallowest direction. To shrink the error a thousandfold rather than by a factor e takes about $\kappa \ln 1000 \approx 6.9\,\kappa$ steps at most.

On the housing data, with the design matrix of Section 3.1, column of ones included:

Design matrix	`np.linalg.cond(X)`	$\kappa = c_{\max}/c_{\min}$, its square
The six features as recorded	654	428,000
The same six, standardised	3.4	11.4
Standardised, plus `area_sqft`	2,286	5.2 million

Read the first two rows together: **standardising divides κ by about 37,000.** The prediction is that iterations shrink by a similar factor, and it can be checked by running gradient descent. Start from zero, use the learning rate $1/c_{\max}$, and stop when the coefficients are within 0.1% of the exact solution of Section 3.1. On the six features as recorded that takes **2.9 million iterations**; the bound above allowed about 3.0 million, so the raw run comes close to the worst case. On the same features standardised it takes **56**, inside a bound of about 79. This is Lesson 2's argument for scaling, arriving from the optimisation side with a number attached.

The third row is Section 3.3's "nearly redundant columns" with a number attached. Adding a column that duplicates another, even with a rounding error between them, multiplies κ by about 460,000. That is why the coefficients in Section 7.3 are unusable while the predictions remain fine:

the ravine is almost perfectly flat along the direction that trades `area_sqm` against `area_sqft`, so the fit has almost no basis for choosing a point along it — but every point along it predicts equally well.

The same number governs the exact solution, which is the tidy part: a large condition number is simultaneously why $(X^\top X)^{-1}$ is untrustworthy and why gradient descent is slow. One quantity, both failure modes. Section 6.4 returns to it with an extreme case, a design matrix whose κ reaches 10^{19}, once the tool that repairs it has been introduced.

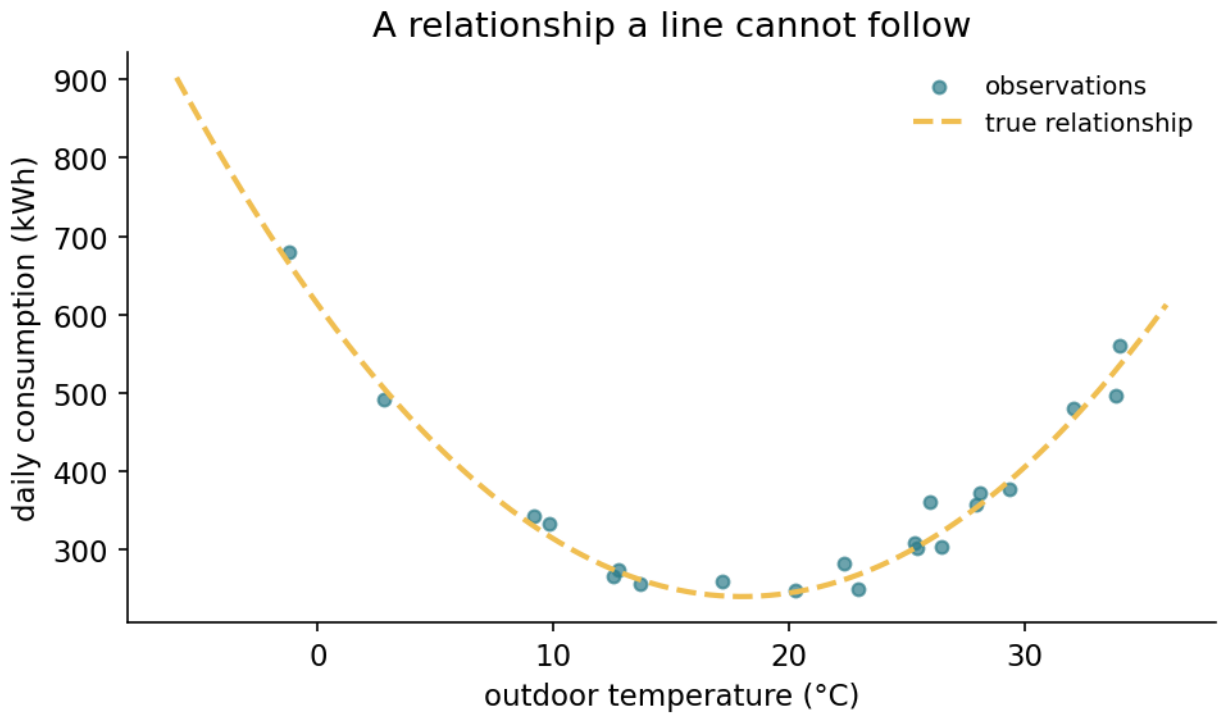
---

5. Curves, and the price of flexibility

5.1 A linear model that bends

"Linear" refers to the coefficients, not the inputs. Nothing prevents us from handing the model x^2 as an additional column:

$$\hat{y} = w_1 x + w_2 x^2 + b$$



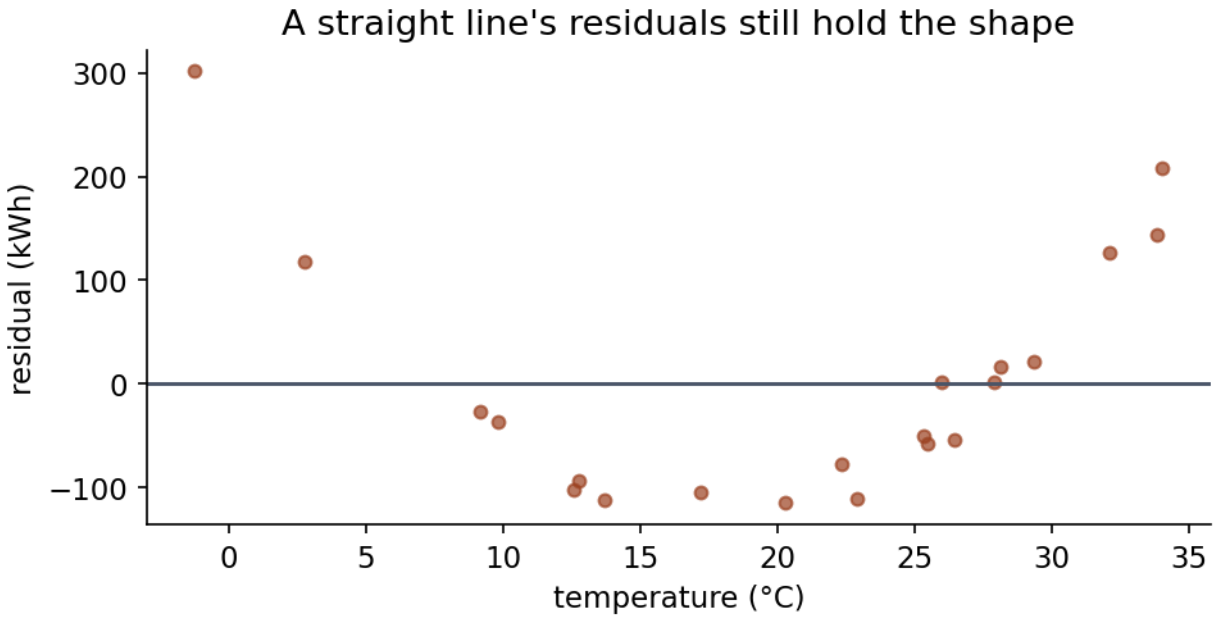
Daily energy against outdoor temperature, with a minimum near 18 °C. Heating below, cooling above: no straight line can follow this, and no amount of fitting will make one.

This is still least squares — the same normal equation, the same code — on a design matrix with an extra column. The model is linear in w, which is all the derivation in Section 3 ever required.

The same trick covers interactions ($x_1 x_2$), logarithms, and any other transformation you can compute. It is the reason linear models remain useful on relationships that are not straight lines.

5.2 What it costs

Notebook 2 fits a curve — daily energy consumption against outdoor temperature, with a minimum around 18 °C — on 21 training observations. Here is the whole lesson in one table, measured with the root mean squared error (RMSE) — the square root of the mean squared error, which puts the number back into the units of the target:



The straight line's residuals still carry the shape of the curve. Residuals that hold a pattern are the model telling you it has not finished.

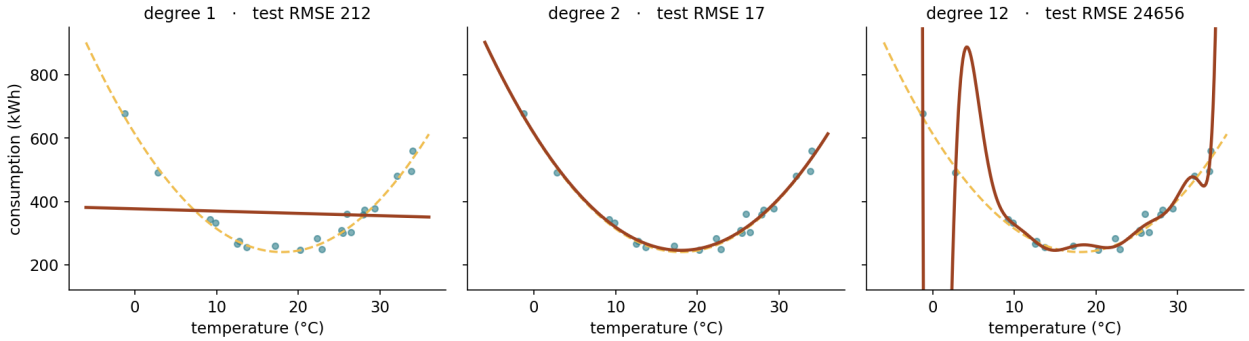
Degree	Training RMSE	Test RMSE
1	113.3	212.1
2	17.6	17.4
3	17.5	**16.9**
6	17.0	23.3
9	16.2	590.1
12	13.6	24,655.7

Training error falls at every step. From 113 down to 16, monotonically, without ever suggesting that anything is going wrong. **Test error falls to degree 3 and then climbs** — by degree 12 it is nearly **1,500 times** worse than the best.

Had we chosen the model by training error, we would have selected degree 12: the worst of the six, and the training error would have congratulated us the whole way. This is Lesson 1's empirical-versus-expected risk argument with numbers in it, and it is why Section 2.3 of Lesson 1's handout matters.

5.3 The mechanism

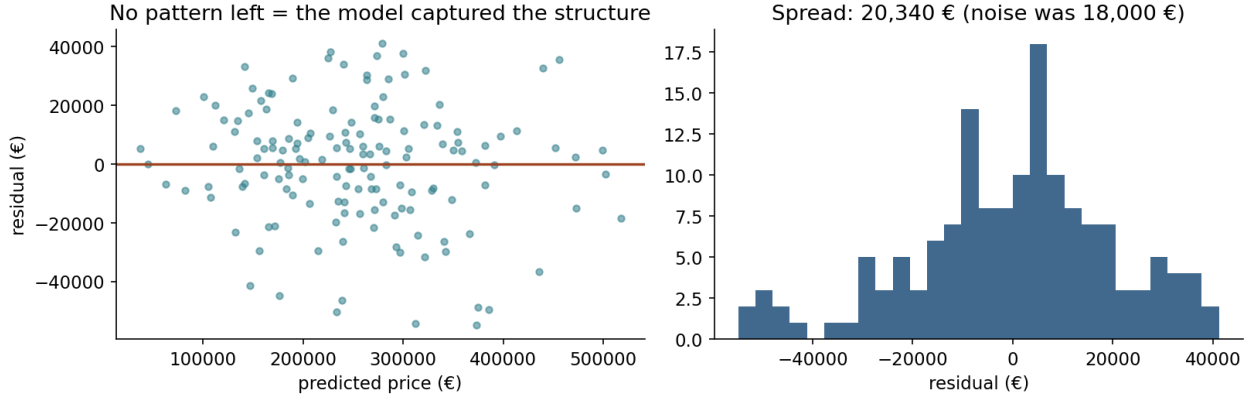
Why does a flexible model behave badly, rather than merely using its flexibility where it is needed? Nothing forces a degree-12 model to wiggle: it contains the degree-2 model as a special case, and could simply set the other ten coefficients near zero. It does not, and the reason is worth seeing before it is explained.



The same 21 observations at three degrees. The rightmost passes closest to the training points, which is precisely why it is the worst model here.

Look at the coefficients. The degree-2 fit has a largest coefficient of 411. The degree-12 fit needs 3,097,038,010 — seven million times larger.

They are large because they work in **opposition**. With 21 points and 12 coefficients there are many ways to pass close to every point, and the arrangement least squares finds involves terms that nearly cancel: one pushing the curve up where the next pushes it down, the cancellation failing exactly where a training point sits. Between the points, nothing constrains the swing.



The right degree leaves residuals with no pattern left in them — scattered around zero, no shape. That is what "nothing left to model" looks like, and it is the check to run on any fit.

This observation is the whole basis of the next section. If large opposing coefficients are the symptom, **charge for coefficient size** and the symptom should disappear.

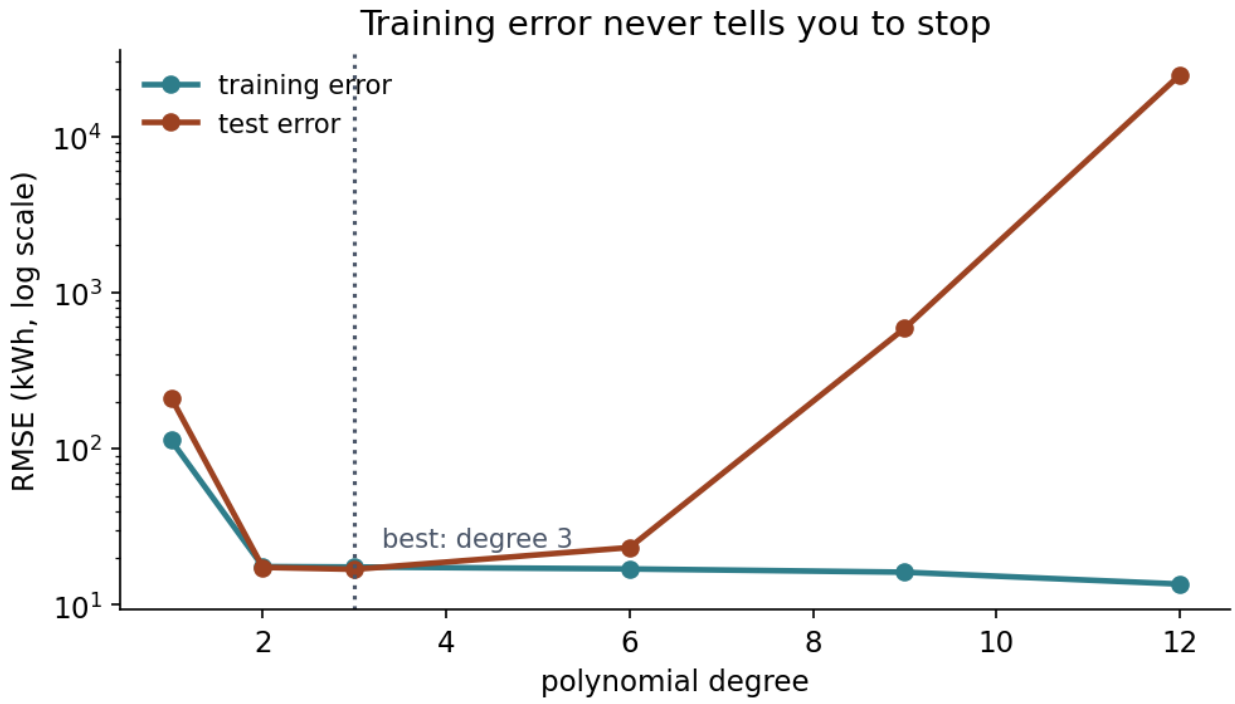
---

6. Regularisation

6.1 Two penalties

Add a price for size to the cost:

$$J_{\text{ridge}}(w) = \text{MSE}(w) + \lambda \sum_{j=1}^{n} w_j^2 \qquad J_{\text{lasso}}(w) = \text{MSE}(w) + \lambda \sum_{j=1}^{n} |w_j|$$



Training error falls with every degree added; test error turns around. The gap between the two curves is the overfitting, and the turning point is what regularisation exists to find without hunting for it by hand.

Ridge (also called L_2 or Tikhonov regularisation) charges the sum of squares; Lasso charges the sum of absolute values. The parameter $\lambda \geq 0$ sets the exchange rate between fitting the data and keeping coefficients small; at $\lambda = 0$ both reduce to ordinary least squares.

A note on names. The literature writes this penalty strength as λ, which is what we use, and reserves α for the learning rate of Section 4. scikit-learn, unhelpfully, calls the penalty parameter `alpha` — so `Ridge(alpha=0.01)` in code is $\lambda = 0.01$ in these pages.

The intercept is never penalised. It is not a claim about any feature, only where the surface sits, and shrinking it would bias every prediction towards zero. Both `Ridge` and `Lasso` in scikit-learn handle this for you.

Scaling is mandatory here, and for a reason that follows directly from the formula: the penalty is a sum over coefficients, so a feature measured in metres and the same feature measured in kilometres attract penalties differing by a factor of a thousand. Lesson 2 introduced scaling for the sake of optimisation speed; here it is a matter of correctness.

6.2 Ridge has a closed form

The picture first. Section 3.3 said least squares fails when two columns carry the same information: the matrix cannot be inverted, because there is a direction in which moving the coefficients changes nothing at all. The problem is flat in that direction, so there is no unique lowest point.

Ridge tilts the floor. By charging for coefficient size it makes every direction cost something, so the flat valley acquires a slope and a single lowest point appears. That is why Ridge always has an answer where least squares has none.

Now the algebra. Ridge, unlike Lasso, can be solved exactly. Differentiating

$$J(\theta) = \frac{1}{2m}\|X\theta - y\|^2 + \lambda\|\theta\|^2$$

gives, by the same steps as Section 3.1 plus $\nabla_\theta(\lambda\theta^\top\theta) = 2\lambda\theta$:

$$X^\top X\theta - X^\top y + 2m\lambda\theta = 0 \qquad \Longrightarrow \qquad \boxed{\theta = (X^\top X + \tilde{\lambda} I)^{-1} X^\top y}$$

writing $\tilde{\lambda} = 2m\lambda$ for brevity. Compare it with the ordinary normal equation: the only change is $\tilde{\lambda} I$ added to the matrix being inverted.

Why there is a factor to keep track of at all. Our cost divides the fit term by $2m$ but not the penalty, so the two are measured on different scales and the $2m$ reappears in the solution. Write the cost as $\|X\theta - y\|^2 + \lambda\|\theta\|^2$ instead — no $1/2m$ — and the solution is exactly $(X^\top X + \lambda I)^{-1} X^\top y$, which is the form most textbooks print.

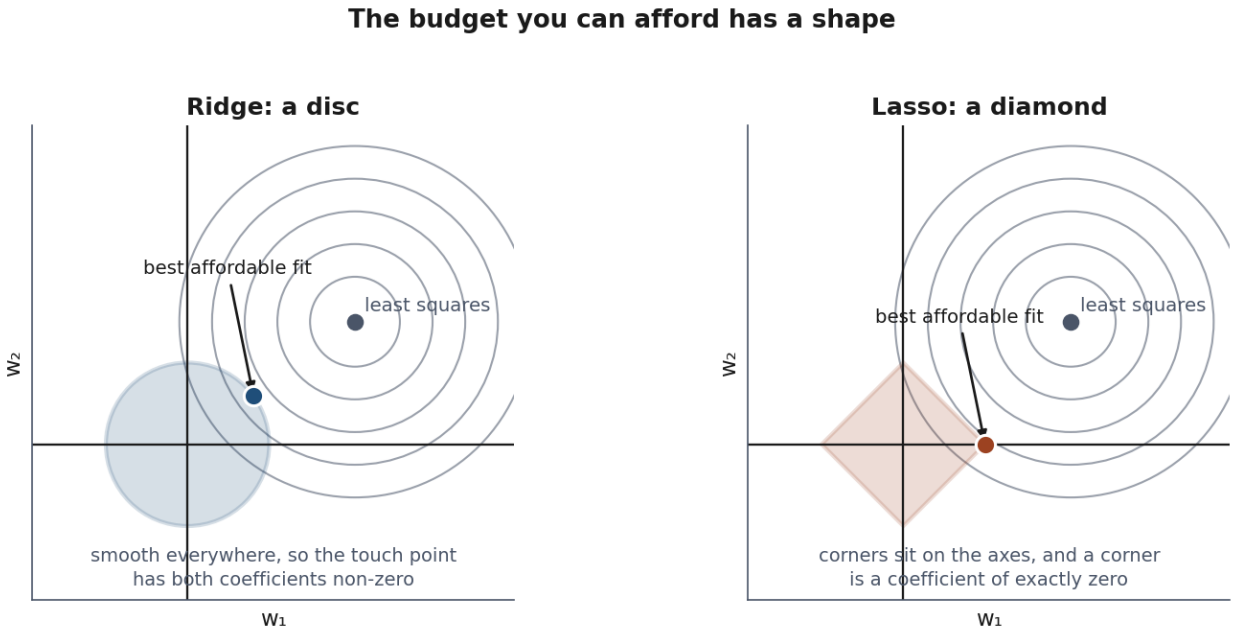
And the intercept. Written with θ, this penalty also charges the intercept, which Section 6.1 said is never penalised. In practice the entry of I that belongs to the intercept is set to zero, or the features and the target are centred first so that the intercept drops out of the problem; scikit-learn does the second. The formula above keeps the full I only to stay short.

Neither convention is more correct, and the models they produce are identical once λ is rescaled. But it does mean **a penalty strength is only meaningful alongside the cost function it belongs to**: `Ridge(alpha=1)` in scikit-learn is not the same number as a λ of 1 taken from a paper, and comparing them directly is a mistake worth not making. This is the same trap as the naming clash in Section 6.1, one level deeper.

That small change is why Ridge is numerically well behaved. $X^\top X$ can be singular — Section 3.3 — but $X^\top X + \tilde{\lambda} I$ never is for $\tilde{\lambda} > 0$: adding a positive constant to the diagonal shifts every eigenvalue up by $\tilde{\lambda}$, so none of them can be zero. Ridge always has a unique solution, even when least squares has none or infinitely many.

6.3 Why Lasso reaches zero and Ridge does not

The picture first. Think of the penalty as a budget. You may spend a fixed total on coefficients, and you want the best fit that money can buy. The two methods differ only in how they charge you — by the square of each coefficient, or by its absolute value — and that changes the *shape* of what you can afford.



The constraint regions: a disc for Ridge, a diamond for Lasso. The diamond has corners ***on the axes****, and a corner is where the solution tends to land — which is the whole reason Lasso produces exact zeros and Ridge does not.*

Draw the affordable region in two dimensions. Charging squares gives a circle; charging absolute values gives a diamond standing on its corners. The best fit you can afford sits where the region first touches the contours of the error.

A circle is smooth, so that contact happens at an ordinary point of its edge, where both coefficients are non-zero. A diamond has **corners, and its corners lie on the axes** — a corner is a point where one coefficient is exactly zero. Corners stick out, so they get touched first.

That is the entire reason Lasso produces zeros and Ridge does not. Now the same thing formally.

Rewrite each as a **constrained** problem: minimise the squared error subject to a budget t on the coefficients. The two forms are equivalent (a result called Lagrange duality): for every penalty strength λ there is a budget t that gives the same solution, and the other way round.

$$\text{Ridge: } \min_w \text{MSE}(w) \;\; \text{s.t. } \sum_j w_j^2 \le t \qquad \text{Lasso: } \min_w \text{MSE}(w) \;\; \text{s.t. } \sum_j |w_j| \le t$$

In two dimensions the ridge constraint region is a **disc**; the lasso region is a **diamond** with corners on the axes. The squared-error contours are ellipses centred on the least-squares solution, and the constrained optimum is where the smallest ellipse first touches the region.

A disc has no corners: the contact point is almost always an ordinary point of its edge, with both coordinates non-zero. A diamond has corners, and its corners sit **exactly on the axes** — a corner is where one coordinate is zero. An expanding ellipse is disproportionately likely to touch a corner first, because the corner protrudes.

So the sparsity of Lasso is not a numerical accident and not a tolerance threshold. It is what happens when a constraint region has corners on the axes.

6.4 Worked: what a penalty buys

Notebook 3 applies Ridge to the degree-12 disaster from Section 5:

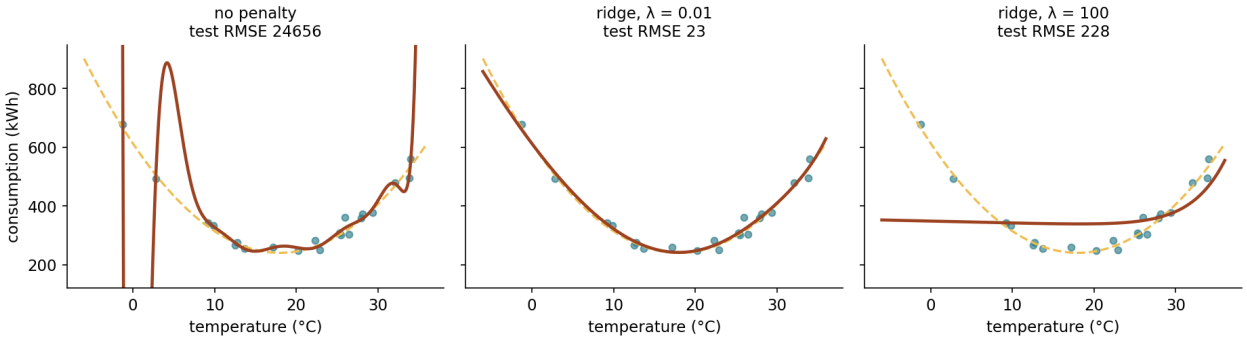


The trade, swept over ten orders of magnitude. Test error falls from 57 at $\lambda = 10^{-8}$ to 14.6 at $\lambda = 10^{-4}$, then climbs to 228. Two things to read off it: the minimum is nowhere near the round number you would have guessed, and the floor is broad — anything between 10^{-6} and 10^{-2} is within a few kWh of the best. That flatness is why a penalty can be searched for rather than solved for, and Lesson 5 is how.

Model	Training RMSE	Test RMSE	Largest |w|
No penalty	13.6	24,655.7	3,097,038,010
Ridge, = 0.01	18.0	**22.8**	365
Ridge, = 1	44.5	105.2	156
Ridge, = 100	96.0	227.6	6

A penalty of $\lambda = 0.01$ — barely a touch — cuts the largest coefficient from 3,097,038,010 to 365 and the test error from 24,656 to 23, which is the noise floor. A thousandfold improvement, from one number.

And notice the training column: it gets worse at every step. That is the trade being made explicitly — we accept a worse fit on the data we have, in exchange for a better fit on data we do not. The exchange stops paying: the test error rises from 22.8 at $\lambda = 0.01$, to 105 at $\lambda = 1$, to 228 at $\lambda = 100$ — a tenfold loss against the best penalty, because the model has become too rigid to follow the curve.



The same three fits drawn on the data, which is the table's last column made visible. Look first at the vertical scale on the left: the unpenalised curve leaves the axes entirely between the training points and returns only where a point is there to pull it back. At $\lambda = 0.01$ the fit is indistinguishable from the true curve (dashed). At $\lambda = 100$ it is a nearly flat line — wrong, but wrong by a few hundred kWh rather than by thousands. That is the asymmetry the table hides: over-penalising is a mild failure, under-penalising is a catastrophic one.

The useful range here spans four orders of magnitude, which is the practical argument for why λ cannot be guessed and must be searched. Lesson 5 provides the machinery.

What else the penalty buys: an answer that does not move. Section 4.4 tied the condition number κ to two failures: slow gradient descent, and an exact solution that cannot be trusted. The degree-12 polynomial of the table above shows what "cannot be trusted" means in practice. After standardising, its design matrix is 21×13 and **full rank**: every column is genuinely independent. But `np.linalg.cond` returns 4.2×10^9, so κ, its square, is 1.8×10^{19}. Nothing is singular, no warning is raised, and the fit is perfectly deterministic: run it three times in the same container and the coefficients agree to the last digit.

Run it on a different linear-algebra backend (the numerical library NumPy calls underneath to solve the system) and the largest coefficient moves from **247,514 to 3,097,038,010**, a factor of **12,500**, on identical code and identical data. Both are correct least-squares answers to the same question, and the question simply does not have a stable answer: the ravine is so flat along one direction that where you stop in it is decided by rounding.

Now compare what the penalty does to that. In scikit-learn's convention, the textbook form of Section 6.2, the matrix Ridge inverts is $X^\top X + \lambda I$. Its eigenvalues are those of $X^\top X$ with λ added to each (Section 6.2), so its condition number is (largest eigenvalue of $X^\top X + \lambda$) divided by (smallest $+ \lambda$). The penalty puts a floor under the smallest eigenvalue, so the ratio can no longer explode.

λ	κ of $X^\top X + \lambda I$
0	1.8×10^{19}
0.01	23,100
1	232
100	3.3

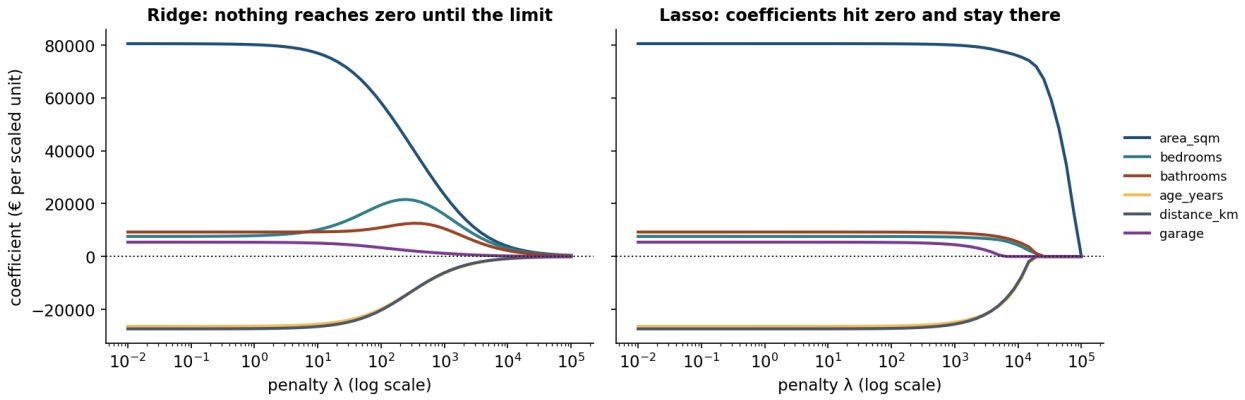
The table at the start of this section is the consequence, and it is worth looking at twice. When that notebook was re-run on a different software installation, **only the unpenalised row moved**; the

three penalised rows came back identical to the digit. A penalty of 0.01, which barely constrains the fit, already buys fifteen orders of magnitude of conditioning.

So regularisation is not only a cure for overfitting. It is also what makes a number reproducible, and that is a reason to reach for it that has nothing to do with generalisation.

6.5 Worked: Lasso as a feature selector

On the housing data, raising λ drops features in a specific order:



Every coefficient tracked as the penalty sweeps from nothing to a lot. Ridge coefficients approach zero and arrive only in the limit; Lasso coefficients reach it at a finite penalty and stay there. What to look at on the left is `bedrooms`*, which* ***triples*** *before it falls: as the penalty pushes* `area_sqm` *down, the feature correlated with it at 0.77 picks up the slack. Shrinkage is not the same thing as each coefficient falling monotonically, and Section 7.2 is why.*

	Features kept
1,000	all six
10,000	five — `garage` dropped
20,000	three — `area`, `bedrooms`, `bathrooms`
40,000	one — `area_sqm`

The order is not arbitrary. `garage` goes first: on a common scale it has the smallest effect of the six. `area_sqm` survives longest because it explains the most price per unit of coefficient spent.

Nobody told Lasso which features mattered. It was told to keep the total size of its coefficients within a budget, and this is the arrangement that buys the most accuracy for that budget. **The selection depends entirely on** λ, which is a choice you must justify — a fact often forgotten when Lasso output is presented as an objective ranking of importance.

---

7. Reading coefficients

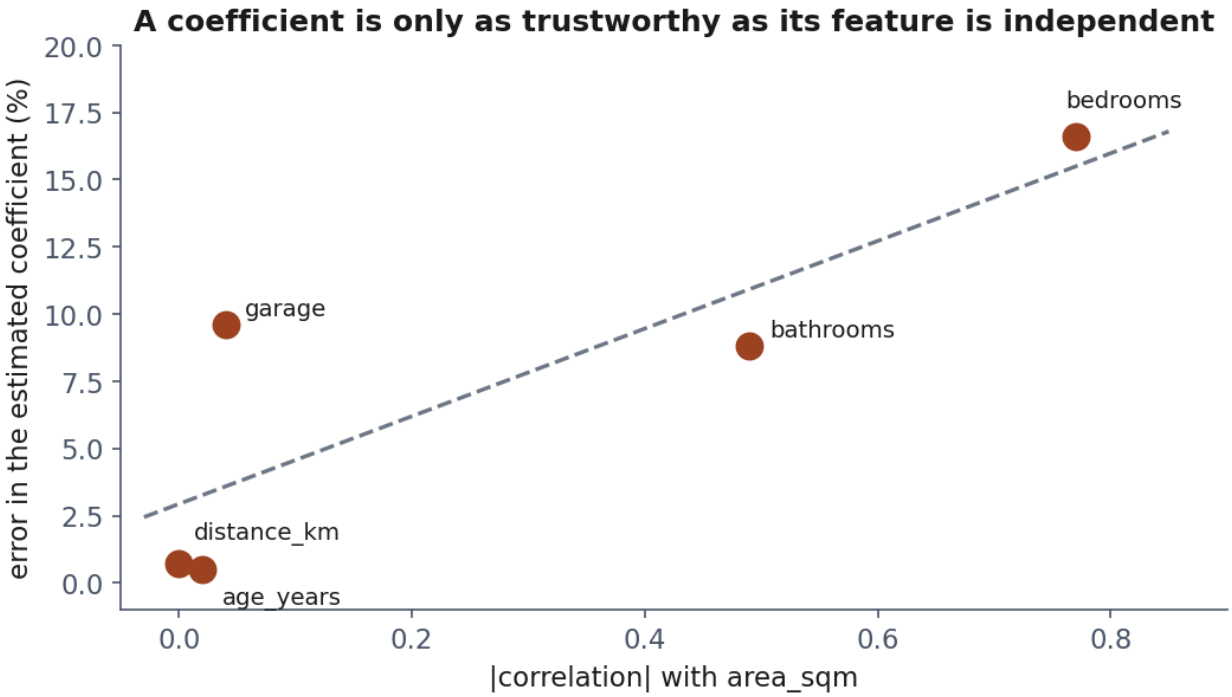
7.1 What a coefficient means, exactly

A coefficient w_j is the change in the prediction for a one-unit change in x_j, **with every other feature held fixed**.

That last clause is doing enormous work and is almost always glossed over. It means the coefficient on `bedrooms` is not "what a bedroom is worth" — it is what a bedroom is worth *to a house whose floor area does not change*, which is a strange object: a house that gains a bedroom without gaining any space.

7.2 Worked: when the estimate can be trusted

Notebook 1 fits all six features and compares against the truth:



The table above, drawn. Each point is one feature: how far its estimated coefficient lands from the truth, against how strongly it correlates with ***`area_sqm`****. Two of the three features that share nothing with area are recovered to within 1%; the two that share a great deal are out by 9% and 17%.* ***`garage`*** *is the honest exception — uncorrelated and still 9.6% out — and it is there to stop the line being read as a law.*

Feature	True	Estimated	Error	Correlation with area
`area_sqm`	2,400	2,421	+0.9%	—
`age_years`	−1,800	−1,791	−0.5%	−0.02
`distance_km`	−6,500	−6,455	−0.7%	0.00
`garage`	12,000	10,850	−9.6%	−0.04
`bathrooms`	14,000	15,229	+8.8%	0.49
`bedrooms`	9,000	7,503	**−16.6%**	**0.77**

Read the first and last columns together. Two of the three features **uncorrelated** with area, `age_years` and `distance_km`, are recovered to within 1%. The two correlated with it are out by 9% and 17%, and the worst estimate belongs to the most correlated feature. `garage` is the exception, uncorrelated and still 9.6% out: a yes-or-no column varies little from house to house, so the 18,000

euros of noise weigh more on its coefficient. Correlation is one source of uncertainty in a coefficient, not the only one.

A coefficient is only as trustworthy as the independence of its feature. With 450 rows and 18,000 euros of noise, this is the precision the data supports — and on a real dataset the same wobble would be there with nothing to reveal it.

7.3 Worked: when it cannot be trusted at all

Notebook 3 adds `area_sqft`, the same measurement in different units (correlation 0.999999). Ordinary least squares on four different random splits of the same data:

Split	`area_sqm`	`area_sqft`
0	9,261	−640
1	14,535	−1,127
2	45,307	−3,989
3	39,061	−3,402

The coefficient on area varies by a factor of five, and the coefficient on its duplicate swings to compensate. Predictions are fine throughout. A square metre is 10.76 square feet, so what one square metre adds is the metres coefficient plus 10.76 times the feet coefficient, and on every split that combined effect stays within about 2% of the true 2,400 (2,424 on the notebook's own split). The explanation is noise.

With Ridge at $\lambda = 10$, the same four splits give roughly 38,000–41,000 on both columns, splitting the effect evenly. These are euros per standard deviation of area, because Ridge runs on scaled features, so they are not comparable with the table's euros per square metre; what matters is that they are stable and equal. Two moderate coefficients cost less under a squared penalty than one huge positive and one huge negative.

So Ridge is not only a cure for overfitting. It is what makes coefficients readable when features overlap — which in real data they nearly always do.

7.4 The warning that belongs on every coefficient

A coefficient describes an association in the training data. It is not a causal effect, and the model has no way to tell the difference.

If houses near the centre are older *and* more expensive, "age" and "distance" will divide the effect between them in whatever proportion best fits the sample. Neither number tells you what would happen if you moved a house.

The professional habit: **state what a coefficient is conditional on**, and be suspicious of any interpretation phrased as an intervention.

---

8. Before the next lesson

1. Work the three notebooks in order. Notebook 1 is the one to do slowly.

2. Read Sections 3.1, 4.1 and 6.3 with a pen — those three derivations carry the lesson, and the exercise assumes them.
3. Take the quiz in `Quizzes/`.
4. **Complete the homework** in `Exercises/03_regression.md`, discussed at the start of Lesson 4, **Friday 23 October 2026**.

Lesson 4 keeps the same machinery and changes the target from a number to a category, which turns out to require a different cost function — and Lesson 4 derives why.

Notation used in this lesson

Symbol	Meaning
x, X	one input, the design matrix
y, $\hat{y}$	true target, prediction
w, b	coefficients, intercept
θ	all parameters together, the intercept included
m, n	number of examples, number of features
J	the cost being minimised
α	the **learning rate**, and nothing else
c, κ	the curvature of the cost in one direction; the ratio of the largest to the smallest
λ	the **regularisation strength** — scikit-learn spells this one `alpha`, which is why the previous row says "and nothing else"
$\bar{x}$, $\bar{y}$	sample means
S_{xx}, S_{xy}	the centred sum of squares and the centred sum of cross-products

Further reading

Resource	Type	Why read it
scikit-learn: linear models	Official docs	The full family, with the mathematics stated compactly
Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, ch. 3	Book	The definitive treatment of linear regression and shrinkage; §3.4 is Ridge and Lasso
Tibshirani, *Regression Shrinkage and Selection via the Lasso* (1996)	Paper	The original, and unusually readable
Hoerl & Kennard, *Ridge Regression* (1970)	Paper	Written to solve the collinearity problem of Section 7.3, not the overfitting one
Gelman & Hill, *Data Analysis Using Regression*, ch. 3–4	Book	On interpreting coefficients honestly, which is Section 7 done properly